

Secure Password Manager

Java Project — Technical Report

Built using AES Encryption and PBKDF2 Hashing

Applied Cryptography — Java Application

March 2026

1. Project Overview

The Secure Password Manager is a Java-based desktop application designed to help users store and manage their credentials securely. The application uses industry-standard cryptographic techniques — AES encryption and PBKDF2 hashing — to ensure that no sensitive data is ever stored or exposed in plain text. It features a Java Swing graphical interface, an auto-lock mechanism, a built-in password generator, and a local SQLite database for offline, cloud-free persistence.

1.1 Objectives

- Provide a single-vault solution protected by a master password
- Hash the master password using PBKDF2 with a unique salt to resist brute-force and rainbow table attacks
- Encrypt every stored credential using AES before writing to the database
- Auto-lock the vault after 30 seconds of inactivity, clearing all in-memory session state
- Generate strong, configurable-length passwords on demand
- Allow users to search, add, delete, and copy credentials from a clean GUI
- Persist all data locally with SQLite — no cloud dependency

1.2 Technology Stack

Component	Technology / Version
Language	Java (SE 11+)
GUI Framework	Java Swing (LoginUI, PasswordManagerGUI)
Encryption	AES (javax.crypto — EncryptionService)
Hashing	PBKDF2WithHmacSHA256 (PasswordHasher)
Database	SQLite (local embedded store)
DB Connector	JDBC (DatabaseConnection)
Password Generator	Custom configurable-length generator

2. Problem Statement

Most users store credentials in browsers or plain-text files — both of which are highly vulnerable. Common weaknesses in existing tools include:

Vulnerability	Description
No encryption	Browsers and plain-text files store passwords with weak or zero encryption, readable by anyone with device access.
No master password	Without a single protected entry point, credentials are accessible to any process or user on the device.
High leak risk	Synced browser vaults and unencrypted files are prime targets for data breaches and credential theft.
No access gates	No auto-lock on inactivity means sessions stay open indefinitely, exposing credentials to unattended access.

Users compensate by reusing weak passwords across services — compounding the risk. The Secure Password Manager directly addresses each of these failure points.

3. System Architecture

The application follows a clean three-layer architecture. Each layer is isolated, independently testable, and replaceable without affecting the others.

Layer	Name	Responsibility
Layer 1	UI Layer	Java Swing handles all user interaction. LoginUI manages master password entry and first-time vault creation. PasswordManagerGUI renders the credential list, search bar, and management controls.
Layer 2	Security Layer	PasswordHasher (PBKDF2), EncryptionService (AES), and PasswordGenerator handle all cryptographic operations. No plaintext ever leaves this layer.
Layer 3	Data Layer	DatabaseConnection uses JDBC and SQLite to store encrypted credentials. No plaintext is ever written to disk.

3.1 Data Flow

- LoginUI captures the master password and passes it to PasswordHasher for PBKDF2 comparison against the stored hash.
- On successful authentication, PasswordManagerGUI renders the decrypted vault.
- When a credential is added, EncryptionService applies AES encryption before any data is passed to DatabaseConnection.
- DatabaseConnection writes only encrypted blobs to SQLite — plaintext never touches the disk.
- The auto-lock timer monitors inactivity; on timeout, all in-memory session state is cleared and LoginUI is shown.

4. Cryptographic Implementation

The password manager uses a two-layer security model to protect both the master password and all stored credentials.

4.1 Layer 1 — Master Password Hashing (PBKDF2)

- The master password is hashed using PBKDF2WithHmacSHA256 with a unique randomly generated salt on every account creation.
- The salt prevents rainbow table attacks — no two hashes are ever identical, even for the same password.
- Only the hash is stored. At login, the entered password is re-hashed with the stored salt and compared. The original password is never stored reversibly.

4.2 Layer 2 — Credential Encryption (AES)

- Every stored password is AES-encrypted before being written to the database.
- Decryption only occurs after successful master password authentication.
- Even with direct database access, an attacker sees only encrypted blobs and irreversible hashes — nothing usable.

Security Mechanism	Protection Provided
PBKDF2 + unique salt	Protects master password from brute-force and rainbow table attacks
AES encryption (per credential)	Ensures stored passwords are unreadable without the master key
Hash-only comparison at login	Master password is never stored in any recoverable form
Encrypted blobs in database	Database compromise yields zero usable credentials

5. Authentication and Storage Flow

5.1 User Login

LoginUI captures credentials. PasswordHasher runs PBKDF2 using the stored salt to produce a hash and compares it to the stored master_hash. On match, the vault is unlocked and control is passed to PasswordManagerGUI.

5.2 Vault Access

PasswordManagerGUI renders the decrypted credential list. DatabaseConnection handles all reads from SQLite, decrypting each record via EncryptionService. The session remains active until the auto-lock timer triggers.

5.3 Add Credential

The user enters a site name, username, and password via the GUI. EncryptionService applies AES encryption to the password before the data leaves memory. The PasswordGenerator is available on demand for generating strong credentials.

5.4 Persist and Display

The encrypted record is written to the SQLite passwords table via DatabaseConnection. PasswordManagerGUI updates the credential list. No plain-text data is ever written to disk at any step.

6. Auto-Lock Mechanism

The vault locks automatically after 30 seconds of user inactivity, protecting unattended sessions from unauthorized access.

Event	Behaviour
Mouse movement or keypress	The 30-second inactivity timer resets to zero
30-second timeout reached	Vault closes immediately; LoginUI is shown
Session cleared on lock	All in-memory session state is wiped — nothing persists
Re-authentication required	Master password must be re-entered to resume access

7. Database Structure

The application uses a local SQLite database with two tables. No plain-text secrets are ever written to disk — every sensitive field is encrypted or hashed before storage.

7.1 settings Table

Column	Type	Purpose
master_hash	TEXT	PBKDF2 hash of the master password — used for login verification
salt	TEXT	Unique random salt used during hashing — prevents rainbow table attacks

7.2 passwords Table

Column	Type	Purpose
website	TEXT	Name of the site or application the credential belongs to
username	TEXT	Account username or email for the site
encrypted_password	TEXT	AES-encrypted credential — unreadable without the master key

8. Key Features

Feature	Description
Master Password	PBKDF2-hashed authentication protects the vault with a single secure key. Never stored in recoverable form.
AES Encryption	All credentials are AES-encrypted before storage — unreadable without the master key.
Auto-Lock	Vault locks automatically after 30 seconds of inactivity, clearing all session state.
Password Generator	Built-in generator with configurable length creates strong, unique passwords on demand.
Credential Management	Search, add, and delete entries. Copy any credential to clipboard directly from the GUI.
SQLite Persistence	JDBC-backed local SQLite database — reliable offline storage with no cloud dependency.

9. Design Patterns & Java Features

Pattern / Feature	Where Used	Benefit
Layered Architecture	UI, Security, Data layers	Clean separation of concerns; each layer independently testable and replaceable
Two-Layer Cryptography	PBKDF2 + AES pipeline	Defence-in-depth — master password and credentials are independently protected
Event-Driven Auto-Lock	Swing event listeners on PasswordManagerGUI	Inactivity detection with timer reset on any user input event
JDBC Abstraction	DatabaseConnection class	Decouples storage technology from business logic; SQLite swappable with minimal change

Pattern / Feature	Where Used	Benefit
Salted Hashing	PasswordHasher (PBKDF2)	Unique salt per account prevents pre-computation and rainbow table attacks
In-Memory Decryption	EncryptionService	Decrypted data held only in RAM during active session; never written to disk

10. Results and Conclusion

The Secure Password Manager successfully delivers a production-ready cryptographic architecture in a Java desktop application. The following outcomes were achieved:

- **AES Encryption:** All credentials are stored in fully encrypted form — zero plain-text data on disk.
- **PBKDF2 Validation:** Master password is verified via secure hash with unique salt; the original password is never stored in any recoverable form.
- **Auto-Lock:** The vault locks on inactivity, blocking unauthorized access to unattended sessions.
- **Clean Architecture:** Security, UI, and data layers are fully separated, demonstrating production-style Java application design.
- **No Cloud Dependency:** Local SQLite storage ensures user data never leaves the device.

The codebase is designed for extensibility — adding new features requires minimal changes to existing classes. The project serves as a strong demonstration of applied cryptography, layered software design, and practical security engineering in Java.

11. Future Enhancements

Enhancement	Description
Cloud Sync	Seamless multi-device access via end-to-end encrypted cloud sync. Passwords stay in sync without ever exposing plaintext.
Biometric Authentication	Unlock the vault using fingerprint or face recognition, replacing the master password for faster, friction-free access.
Clipboard Auto-Clear	Copied passwords are automatically wiped from the clipboard after a configurable timeout, reducing accidental exposure.
Browser Extension	A lightweight extension for autofill and one-click login across Chrome, Firefox, and Edge.
Two-Factor Authentication (2FA)	Add TOTP-based 2FA as an additional layer of protection before vault access is granted.

Enhancement	Description
Unit & Integration Tests	JUnit 5 tests for all cryptographic service methods, with Mockito mocking for database isolation.

12. Project Summary

Metric	Detail
Core Java Classes	6+ (LoginUI, PasswordManagerGUI, PasswordHasher, EncryptionService, PasswordGenerator, DatabaseConnection)
Architecture Layers	3 (UI, Security, Data)
Encryption Algorithm	AES (Advanced Encryption Standard)
Hashing Algorithm	PBKDF2WithHmacSHA256 with unique salt per account
Database	SQLite via JDBC — 2 tables (settings, passwords)
Auto-Lock Timeout	30 seconds of inactivity
Plain-text data on disk	Zero — all credentials AES-encrypted before storage
Cloud dependency	None — fully local, offline-first storage